

PENETRATION TEST WALKTHROUGH

Hidden Keys

스टे가노그래피 정보 추출과 2FA 시크릿 노출을 통한 인증 우회 (Steganographic Data Exfiltration & TOTP Secret Disclosure)

TARGET problem08
PREPARED FOR 8llow8llowmi Team

SEVERITY: MEDIUM

CWE-200 · CWE-522

STATUS: CONFIRMED

1. 개요 (Executive Summary)

problem08(관리자 로그인 문제)을 살펴본 결과, 관리자(8llowMe) 계정의 아이디와 2단계 인증(2FA)에 쓰이는 비밀 키가 메인 페이지 이미지 파일 속에 몰래 숨겨져 있는 것을 발견했다.

이미지는 겉보기엔 평범해 보이지만, 사진 파일에 딸린 부가정보(메타데이터)를 열어보니 숨겨진 정보를 어떻게 뽑아내는지 알려주는 힌트가 그대로 남아있었다. 이 힌트대로 간단한 파이썬 스크립트를 짜서 이미지 픽셀 값을 분석하니, 숨겨진 아이디와 2FA 비밀 키를 바로 뽑아낼 수 있었다. 이렇게 알아낸 아이디로 로그인 시도했고, 비밀번호는 몰랐지만 비밀번호 찾기에 있는 보안 질문에 답해서 새 비밀번호로 바꿀 수 있었다. 마지막으로 이미지에서 같이 찾아낸 2FA 비밀 키를 OTP 앱에 등록하니, 진짜 관리자 계정 정보를 하나도 몰랐음에도 로그인부터 2차 인증까지 전부 통과할 수 있었다.

2. 공격 시나리오 (Attack Narrative)

단계	기법	확보한 정보 / 결과
1	이미지 파일 메타데이터 분석	1.png EXIF Description에서 LSB 추출 규칙(HTML 주석 아님), 관리자 계정 ID노출 확인
2	이미지 다운로드 및 LSB 추출	픽셀 R→G→B 최하위 비트 순차 추출용 스크립트 작성
3	정보 복원 및 조합	로그인 아이디(8llowMe)와 2FA 시크릿 키 조각 6개 확보, 파일명 순서로 조합

4	비밀번호 재설정 기능 오남용	보안 질문(강사 hostname) 답변으로 신규 비밀번호 설정, 계정 탈취 아님에도 로그인 획득
5	2FA 시크릿 등록	확보한 시크릿 키를 OTP 생성기에 수동 등록, TOTP 코드 자체 생성 가능
6	인증 절차 완전 우회	로그인부터 2FA까지 전 구간 통과

3. 상세 재현 절차 (Proof of Concept)

3.1 메인 페이지 이미지 분석 및 힌트 확인

메인 페이지(index.php)에는 이미지 6장과 로그인 버튼만 배치되어 있으며, 페이지 소스 보기(HTML)에는 스테가노그래피 관련 정보가 전혀 노출되지 않는다. 이미지 파일을 다운로드하여 exiftool로 메타데이터를 확인한 결과, 1번 이미지(1.png)의 Description 항목에 LSB 추출 규칙이 평문으로 기록되어 있는 것을 발견하였다: 픽셀 RGB 채널의 최하위 비트를 R→G→B 순서로 추출, 8비트마다 1문자로 변환, 종료 마커 (111111111111110) 확인 시 추출 종료.

```
blue@blue:/var/www/html/wargame/problems/problem04$ exiftool -Description -b /var/www/html/wargame/problems/problem08/assets/images/1.png
oh wait forgot to remove this, quick notes on the stego rule before i forget

(btw there's an ID hint in the picture too - If you look at the picture, there is an ID.)

only the last bit (LSB) of each RGB channel
order is top-left to bottom-right, R -> G -> B
8 bits = 1 char
stop when you hit 111111111111110

don't forget this time, remove before deploy
```

3.2 LSB 추출 스크립트 작성 및 실행

확인한 규칙에 따라 Pillow 라이브러리를 이용한 디코딩 스크립트를 작성하였다.

```
from PIL import Image

def decode_lsb(img_path):
    img = Image.open(img_path).convert('RGB')
    pixels = list(img.getdata())

    bits = ""
    for pixel in pixels:
        for value in pixel: # r, g, b
            bits += str(value & 1)
```

```

end_marker = '1111111111111110'
end_idx = bits.find(end_marker)
if end_idx != -1:
    bits = bits[:end_idx]

# 8비트씩 끊어서 문자로 변환
message = ""
for i in range(0, len(bits) - len(bits) % 8, 8):
    byte = bits[i:i+8]
    message += chr(int(byte, 2))

return message

if __name__ == '__main__':
    result = decode_lsb('1.png')
    print(result)

```

3.3 아이디 및 시크릿 키 조각 조합

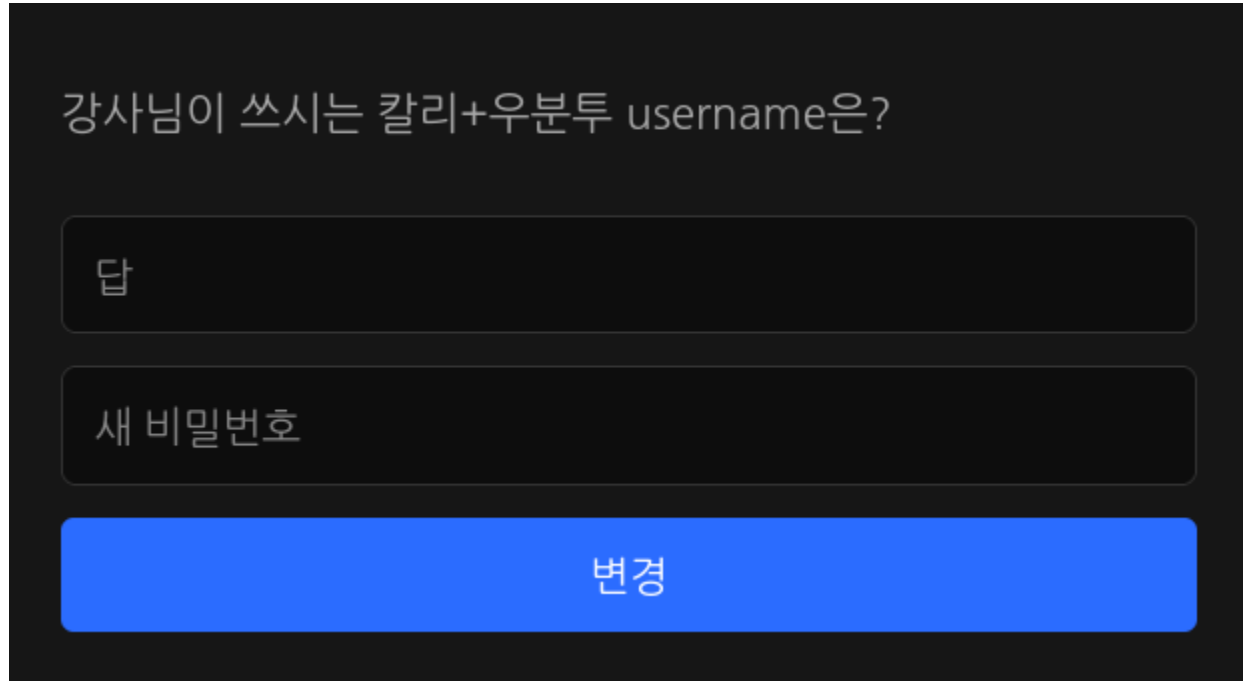
1번 이미지에서는 로그인 아이디와 시크릿 키 첫 조각이 구분자로 함께 추출되었고, 2~6번 이미지에서는 나머지 조각이 순차 추출되었다. 파일명(1.png~6.png) 순서대로 조각을 이어 붙여 완전한 시크릿 키를 복원하였다.

파일명	추출 결과
1.png	UZ
2.png	3RB
3.png	2TW
4.png	2ZI
5.png	G2X
6.png	XO

조합 결과 — 시크릿 키: UZ3RB2TW2ZIG2XXO

■ 3.4 비밀번호 재설정을 통한 로그인 획득

복원한 아이디로 로그인을 시도하였으나 초기 비밀번호를 알 수 없어 실패하였다. 비밀번호 찾기 기능으로 이동하여 획득한 아이디를 이용해 노출된 보안 질문에 답변함으로써 임의의 새 비밀번호로 재설정할 수 있었다. 답은 redhappy



강사님이 쓰시는 칼리+우분투 username은?

답

새 비밀번호

변경

■ 3.5 2FA 시크릿 키 등록 및 TOTP 코드 생성

재설정된 비밀번호로 로그인하면 해당 계정에 한해 2단계 인증 절차로 리다이렉트된다. 2차인증을 위해 Google Authenticator 를 스마트폰에 설치 후 오른쪽 하단 플러스 버튼을 누르고 설정 키 입력을 눌러 3.3에서 확보한 시크릿 키(UZ3RB2TW2ZIG2XXO)를 OTP 생성기 애플리케이션에 수동으로 등록(계정이름은 아무거나 입력 후 내 키에 UZ3RB2TW2ZIG2XXO 가 적어야 함) 하여, 정상 사용자와 동일한 방식으로 시간 기반 6자리 인증 코드를 직접 생성할 수 있었다.

< 뒤로

계정 세부정보 입력

계정 이름

flowme

내 키

UZ3RB2TW2ZIG2XXO



flowme

113 939



■ 3.6 인증 완료 및 플래그 확보

생성된 6자리 코드를 2차 인증 페이지에 입력하여 인증에 성공하였으며, 정상 관리자 계정 정보를 사전에 전혀 알지 못한 상태에서 로그인부터 2FA까지 전 구간 인증을 완료하고 플래그를 획득하였다.

FLAG{NThtNGExc285PW4kciQ}

4. 근본 원인 분석 (Root Cause)

- 2FA 시크릿 키와 같은 민감한 인증 비밀값이 별도 보호 없이 정적 이미지 파일 내부에 저장, 배포됨
- 스테가노그래피 추출 규칙이 이미지 파일의 EXIF 메타데이터에 평문으로 기록되어, 파일 자체 분석만으로 정보 은닉 우회 가능
- 비밀번호 재설정에 사용되는 보안 질문의 답변 노출 범위가 넓어(강사 개인 정보 성격), 예측 또는 사회공학적 접근에 취약
- 계정 소유자의 사전 지식 없이도 재설정 기능만으로 계정 제어권 획득이 가능한 구조